CORPORACIÓN UNIVERSITARIA ANTONIO JOSÉ DE SUCRE

ASIGNATURA: SISTEMAS DISTRIBUIDOS

PROYECTO INTEGRADOR
UAJS SMART CAMPUS
ENTREGABLE 2

Persistencia de datos, CRUD completos e integración entre microservicios

Guía de trabajo, implementación técnica, pruebas, seguimiento en ClickUp y trazabilidad con GitHub

Corporación Universitaria Antonio José de Sucre (UNIAJS)
Asignatura: Sistemas Distribuidos
Docente: Laura Paternina Álvarez
Año: 2026

Corporación Universitaria
Antonio José de Sucre

Todos los equipos tienen el Entregable 1 funcionando y aprobado. No se repetirán sus actividades; esta guía inicia directamente con el modelo de datos, persistencia, CRUD, integración y pruebas.

Control del documento

Campo	Descripción
Nombre	Proyecto Integrador UAJS Smart Campus – Entregable 2
Versión	1.0
Propósito	Orientar la segunda etapa de construcción del prototipo distribuido.
Prerrequisito	Entregable 1 revisado: frontend React, API Gateway, seis microservicios con /health, endpoints iniciales y repositorio organizado.
Resultado esperado	Aplicación con persistencia real, operaciones CRUD completas, validaciones, integración estable y pruebas documentadas.
Continuidad	El Entregable 3 será la entrega final: integración avanzada, endurecimiento de seguridad, despliegue completo, demostración y videomemoria.

Contenido

1. Organización del proyecto en tres entregables
2. Presentación, propósito y objetivos del Entregable 2
3. Alcance, exclusiones y condiciones de partida
4. Arquitectura objetivo y decisiones técnicas
5. Plan de trabajo por fases
6. Modelo de datos y persistencia
7. Especificación de microservicios y API REST
8. Integración entre servicios y manejo de errores
9. Requisitos de calidad, seguridad y pruebas
10. GitHub, ClickUp y trabajo colaborativo
11. Entregables y estructura del ZIP
12. Rúbrica propuesta y lista de comprobación
13. Proyección hacia el Entregable 3
14. Anexos y plantillas

Corporación Universitaria Antonio José de Sucre

1. Organización del proyecto en tres entregables

El proyecto UAJS Smart Campus se desarrollará en tres entregas progresivas. Cada una debe producir resultados verificables y servir como base para la siguiente. No se debe reemplazar ni rehacer innecesariamente el trabajo aprobado en etapas anteriores; se debe versionar, ampliar y mejorar.

Entrega	Enfoque principal	Resultados mínimos
Entregable 1 – Base funcional y arquitectónica	Diseño funcional, frontend React, backend base, API Gateway y estructura inicial de microservicios.	Vistas principales, componentes y rutas React, lógica inicial, seis servicios creados con /health, dos integraciones iniciales, repositorio GitHub y tablero ClickUp.
Entregable 2 – Persistencia, CRUD e integración	Modelo de datos, persistencia real, CRUD de seis dominios, validaciones, integración y pruebas.	Persistencia real; operaciones CRUD en recursos priorizados; APIs documentadas; integración frontend–Gateway–servicios–BD; errores y validaciones; pruebas con evidencias.
Entregable 3 – Integración avanzada y entrega final	Consolidación distribuida, seguridad, despliegue, monitoreo, documentación final y sustentación.	Sistema integrado y desplegado; configuración segura; pruebas integrales; evidencias finales; README y manuales; videomemoria de máximo 10 minutos; paquete final reproducible.

Importante: el Entregable 2 no es la entrega final. Debe dejar una solución persistente y comprobable, pero puede mantener para el Entregable 3 actividades de despliegue definitivo, observabilidad avanzada, integración ampliada, ajustes de seguridad y videomemoria final.

2. Presentación del Entregable 2

El Entregable 2 corresponde a la segunda etapa de desarrollo de UAJS Smart Campus. Para esta guía se asume que todos los equipos tienen el Entregable 1 funcionando y aprobado. Por tanto, no deberán repetir el diseño inicial, la creación de componentes React, la configuración básica del API Gateway ni la creación de los seis microservicios. El trabajo se concentrará en la persistencia real de datos, los CRUD completos, las reglas de negocio, la integración y las pruebas.

La solución deberá demostrar que cada componente tiene responsabilidades delimitadas y que el frontend no accede directamente a la base de datos. Las operaciones se realizarán a través de APIs REST y los servicios responsables de cada dominio.

2.1 Objetivo general

Implementar la persistencia de datos y las operaciones CRUD de UAJS Smart Campus, fortaleciendo la comunicación entre el frontend React, el API Gateway, los microservicios y sus mecanismos de almacenamiento, mediante APIs REST documentadas, validaciones, manejo de errores y pruebas funcionales.

2.2 Objetivos específicos

- Revisar y ajustar la arquitectura y el código heredados del Entregable 1, conservando su trazabilidad.
- Diseñar el modelo lógico y físico de datos para los dominios de usuarios, solicitudes, reservas, recursos, eventos y notificaciones.
- Configurar una base de datos relacional y su conexión segura desde los servicios que la requieran.
- Implementar operaciones de creación, consulta, actualización y eliminación en los endpoints definidos para cada microservicio.
- Agregar validaciones de entrada, respuestas HTTP coherentes y manejo centralizado de errores.
- Integrar el frontend con los servicios a través del API Gateway, eliminando progresivamente los datos mock de los flujos que ya tengan persistencia.
- Verificar las reglas de negocio de solicitudes, reservas, recursos y notificaciones.
- Documentar las APIs, el modelo de datos, la instalación y la ejecución del sistema.
- Ejecutar pruebas unitarias o de componentes, pruebas de API y pruebas de integración, registrando resultados y evidencias.
- Mantener la trazabilidad individual y grupal mediante ClickUp, ramas, commits y Pull Requests en GitHub.

2.3 Competencias que se evidenciarán

Diseño de una solución distribuida con responsabilidades separadas.
Comunicación cliente-servidor mediante HTTP/HTTPS, APIs REST y JSON.
Persistencia y consistencia básica de datos.
Diseño y consumo de operaciones CRUD.
Validación de reglas de negocio y tratamiento de fallos.
Pruebas técnicas y documentación reproducible.
Colaboración técnica con control de versiones y trazabilidad individual.

3. Alcance, exclusiones y condiciones de partida

3.1 Alcance obligatorio

Conservar el frontend React y las rutas implementadas en el Entregable 1, realizando ajustes necesarios.
Configurar la base de datos relacional elegida y documentar la razón de su elección.
Implementar persistencia para las entidades principales.
Completar CRUD en los microservicios y recursos priorizados.
Conectar el frontend a través del API Gateway; el navegador no debe conectarse directamente a la base de datos.
Validar datos y reglas de negocio antes de almacenar o modificar información.
Responder con JSON y códigos HTTP apropiados.
Probar endpoints, integración y persistencia real después de reiniciar los servicios.
Documentar instalación, variables de entorno de ejemplo, endpoints y resultados de pruebas.
Conservar datos ficticios o de prueba.

3.2 Fuera del alcance obligatorio de esta entrega

Integración con sistemas institucionales reales o acceso a datos personales institucionales.
Despliegue definitivo de todos los componentes en producción, salvo que la docente lo solicite expresamente.
Alta disponibilidad empresarial, balanceo avanzado, Kubernetes o tolerancia a fallos de nivel productivo.
Implementación obligatoria de RabbitMQ, Redis, Elasticsearch o API Gateway comercial; pueden proponerse como mejoras si se justifican.
Videomemoria final completa, que se reserva para el Entregable 3.

3.3 Requisitos previos

El repositorio del Entregable 1 debe estar accesible y contener instrucciones de ejecución.
Los seis microservicios deben existir y responder a /health.
El API Gateway debe tener rutas iniciales para users, requests, reservations, resources, events y notifications.
El frontend debe contar con vistas base y los flujos iniciales de solicitudes y reservas.
El equipo debe registrar en ClickUp los defectos o pendientes heredados antes de iniciar nuevas funcionalidades.

3.4 Regla de continuidad

El Entregable 1 se considera aprobado y operativo. Solo se permite una revisión breve de salud al comenzar: comprobar que el frontend, el API Gateway y los seis microservicios siguen ejecutándose. Si aparece una regresión que impida desarrollar el Entregable 2, debe registrarse y corregirse sin convertir esta entrega en una repetición de la primera.

4. Arquitectura objetivo del Entregable 2

La arquitectura lógica de referencia será:

Usuario → Frontend React → API Gateway → Microservicio de dominio → Base de datos

Cada microservicio será responsable de su lógica de negocio. Para mantener la separación, el frontend solo consumirá las rutas públicas del Gateway. No se permite conectar React directamente a MySQL ni concentrar toda la lógica de los seis dominios en un único backend monolítico.

4.1 Tecnologías recomendadas

Capa	Tecnología base	Criterio
Frontend	React, JavaScript, React Router, Fetch o Axios	Reutilizar la solución del Entregable 1.
API Gateway	Node.js y Express	Enrutamiento, control de errores y punto de entrada.
Microservicios	Node.js y Express, salvo decisión técnica aprobada	Un servicio por dominio funcional.
Persistencia	MySQL o motor relacional equivalente	Debe documentarse la elección y la configuración.
Comunicación	REST sobre HTTP en local; HTTPS cuando el entorno lo permita; JSON	Contratos explícitos y respuestas coherentes.
Pruebas API	Postman o herramienta equivalente	Colección exportable y resultados verificables.
Versionamiento	Git y GitHub	Ramas, commits, PR y revisión.
Seguimiento	ClickUp	Tareas, responsables, estados y evidencias.

4.2 Organización de carpetas sugerida

La estructura puede ajustarse al repositorio existente, siempre que conserve una separación clara entre frontend, Gateway y servicios.

```
ua-s-smart-campus/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── hooks/
│   │   ├── services/       # clientes HTTP del frontend
│   │   ├── routes/
│   │   └── styles/
│   └── README.md
├── backend/
│   ├── api-gateway/
│   └── services/
│       ├── users-service/
│       ├── requests-service/
│       ├── reservations-service/
│       ├── resources-service/
│       ├── events-service/
│       └── notifications-service/
├── database/
│   ├── schema.sql
│   ├── seed.sql
│   └── README.md
├── docs/
│   ├── arquitectura/
│   ├── modelo-datos/
│   ├── api/
│   └── pruebas/
├── postman/
├── .env.example
└── README.md
```

5. Plan de trabajo por fases

Fase 1. Diseño definitivo del modelo de datos

Actividades: definir entidades, atributos, claves primarias y foráneas, cardinalidades, restricciones y reglas de integridad para usuarios, roles, solicitudes, reservas, recursos, eventos y notificaciones.
Evidencias: diagrama entidad-relación, diccionario de datos, decisiones sobre propiedad de datos por microservicio y tareas técnicas en ClickUp.
Criterio de aceptación: el modelo de datos representa los procesos del sistema, sus relaciones están justificadas y cada entidad tiene campos y restricciones documentados.

Fase 2. Configuración de persistencia

Actividades: configurar MySQL o un motor relacional equivalente; definir variables de entorno; crear scripts o migraciones y preparar datos semilla ficticios.
Evidencias: scripts reproducibles de creación y carga, archivo .env.example sin secretos, evidencia de conexión y documentación de inicialización.
Criterio de aceptación: la base de datos puede crearse desde cero siguiendo las instrucciones y los servicios acceden a ella sin credenciales expuestas.

Fase 3. CRUD de Usuarios y Recursos

Actividades: implementar crear, listar, consultar por identificador, actualizar y eliminar o desactivar usuarios y recursos, con validaciones y persistencia real.
Evidencias: endpoints documentados, pruebas Postman, respuestas HTTP, capturas de registros almacenados y evidencia desde la interfaz cuando aplique.
Criterio de aceptación: las operaciones CRUD funcionan sobre la base de datos; los campos obligatorios y las restricciones son validados en el backend.

Fase 4. CRUD de Solicitudes

Actividades: implementar las operaciones CRUD de solicitudes; generar identificadores; persistir fechas, descripción, prioridad, responsable y estado; validar transiciones permitidas.
Evidencias: endpoints, pruebas de transiciones válidas e inválidas, consulta del historial/estado disponible y evidencia de persistencia.
Criterio de aceptación: una solicitud se registra con estado inicial REGISTRADA y el backend impide transiciones no autorizadas o no definidas.

Fase 5. CRUD de Reservas

Actividades: implementar alta, consulta, modificación y cancelación de reservas; validar existencia de usuario y recurso, rango de fechas y disponibilidad.
Evidencias: endpoints, pruebas de reserva válida, conflicto de horario, modificación y cancelación.
Criterio de aceptación: el backend rechaza fechas inválidas y solapamientos de reservas activas para el mismo recurso.

Fase 6. CRUD de Eventos y Notificaciones

Actividades: completar las operaciones CRUD de eventos y notificaciones; implementar marcado de notificaciones como leídas y validación de los campos obligatorios.

Evidencias: endpoints, pruebas Postman, registros persistidos y capturas de las vistas correspondientes.
Criterio de aceptación: los eventos y las notificaciones se crean, consultan y actualizan correctamente y la información persiste tras reiniciar los servicios.

Fase 7. Integración del API Gateway y comunicación entre servicios

Actividades: revisar rutas públicas del Gateway, integrar llamadas entre servicios cuando una regla de negocio lo requiera y evitar el acceso directo a tablas de otros dominios.
Evidencias: diagrama de secuencia o flujo, colección Postman y pruebas de integración de cada comunicación relevante.
Criterio de aceptación: el Gateway enruta correctamente y las comunicaciones entre servicios están documentadas, controlan fallos y respetan la propiedad de los datos.

Fase 8. Integración del frontend con las APIs persistentes

Actividades: sustituir los datos mock en los flujos completados por peticiones al Gateway; conectar formularios y listas con operaciones persistentes; mostrar carga, éxito, errores y estados vacíos.
Evidencias: capturas y video corto o secuencia de imágenes de los flujos, más pruebas que relacionen la acción de React con el registro en la base de datos.
Criterio de aceptación: los flujos principales funcionan desde React a través del Gateway y muestran información recuperada desde la persistencia real.

Fase 9. Pruebas funcionales, de integración y regresión

Actividades: Ejecutar pruebas de endpoints, validaciones, persistencia, errores y flujos completos.
Evidencias: Plan y matriz de pruebas; colección Postman; registro de defectos y correcciones.
Criterio de aceptación: Las pruebas críticas pasan y los fallos conocidos quedan documentados.

Fase 10. Documentación técnica y cierre del Entregable 2

Actividades: Actualizar README, documentación de APIs, modelo de datos, ClickUp y evidencias de GitHub.
Evidencias: PDF técnico; README; diagramas; capturas; enlaces y matriz individual.
Criterio de aceptación: Una persona externa al equipo puede instalar, ejecutar y comprobar la solución con la guía.

6. Modelo de datos y persistencia

El equipo debe diseñar un modelo relacional coherente con los seis dominios del proyecto. Las tablas pueden variar según la decisión técnica, pero las entidades mínimas recomendadas se presentan a continuación. El equipo debe justificar cualquier cambio.

Entidad sugerida	Atributos mínimos orientativos	Reglas y relaciones
Users	id, name, email, role_id, status, created_at, updated_at	Correo único; role_id referencia roles; no guardar contraseñas en texto plano.
Roles	id, name, description	Valores de prueba: ESTUDIANTE, DOCENTE, ADMINISTRATIVO, ADMIN.
Resources	id, code, name, type, location, status, description, created_at, updated_at	Código único; estado y tipo controlados.
Requests	id, code, user_id, service_type, department, description, priority, status, assigned_to, created_at, updated_at	Código único; user_id referencia users; estado restringido a valores definidos.
Reservations	id, user_id, resource_id, start_at, end_at, status, purpose, created_at, updated_at	Referencias a users y resources; end_at debe ser posterior a start_at; validar conflictos.
events	id, title, description, event_date, start_time, end_time, location, status	Fechas y horas válidas; estado definido.
notifications	id, user_id, title, message, type, read_at, created_at, related_entity, related_id	Puede relacionarse con usuarios y con una entidad del sistema.

Corporación Universitaria
Antonio José de Sucre

6.1 Requisitos del modelo

Definir una clave primaria para cada tabla.

Usar claves foráneas cuando exista una relación entre entidades del mismo esquema.

Definir restricciones NOT NULL, UNIQUE y CHECK cuando el motor las soporte.

Elegir tipos de datos apropiados para identificadores, textos, estados, fechas y horas.

Evitar almacenar listas complejas o entidades relacionadas en una sola columna sin justificación.

Documentar cardinalidades, reglas de eliminación y comportamiento ante registros relacionados.

Crear datos de prueba ficticios, suficientes para demostrar los flujos principales.

Entregar scripts reproducibles para crear la estructura y poblarla.

6.2 Separación de bases de datos en microservicios

En una arquitectura de microservicios, cada servicio debe ser dueño de sus datos. Para una entrega académica se puede utilizar una misma instancia de MySQL con esquemas o

bases separadas por servicio, siempre que la propiedad de los datos esté documentada. No se debe permitir que un servicio modifique directamente las tablas internas de otro servicio. Si el equipo opta por una única base de datos compartida por razones de tiempo, deberá declararlo como decisión transitoria, limitar el acceso por servicio y describir cómo se separaría en una evolución futura. Esta decisión no debe presentarse como aislamiento completo de datos.

6.3 Datos semilla y reproducibilidad

- Crear un script schema.sql o migraciones que creen las tablas y restricciones.
- Crear seed.sql con usuarios, roles, recursos, solicitudes, reservas, eventos y notificaciones ficticias.
- Documentar el orden de ejecución de los scripts.
- Comprobar que la aplicación puede consultarse después de reiniciar los servicios.
- No incluir información real de estudiantes, docentes o funcionarios ni credenciales institucionales.

7. Especificación de microservicios y API REST

Los endpoints siguientes son una guía de contrato. Los equipos pueden usar rutas equivalentes si mantienen consistencia y las documentan. El Gateway debe publicar las rutas de acceso y dirigir cada petición al servicio correspondiente.

Servicio	Responsabilidad	Rutas públicas sugeridas a través del Gateway
Usuarios	Administrar usuarios, roles y datos de perfil ficticios.	GET/POST /api/users; GET/PUT/DELETE /api/users/:id
Solicitudes	Registrar y consultar solicitudes, prioridad y estado.	GET/POST /api/requests; GET/PUT/DELETE /api/requests/:id
Reservas	Gestionar reservas de espacios y recursos.	GET/POST /api/reservations; GET/PUT/DELETE /api/reservations/:id
Recursos	Administrar salas, laboratorios, equipos y otros recursos.	GET/POST /api/resources; GET/PUT/DELETE /api/resources/:id
Eventos	Administrar eventos y actividades institucionales ficticias.	GET/POST /api/events; GET/PUT/DELETE /api/events/:id
Notificaciones	Crear, consultar y marcar notificaciones como leídas.	GET/POST /api/notifications; GET/PUT/DELETE /api/notifications/:id

7.1 Operaciones CRUD

Operación	Método HTTP	Resultado esperado	Código habitual
Listar	GET /recurso	Devuelve colección JSON; puede aceptar filtros y paginación básica.	200 OK
Consultar por ID	GET /recurso/:id	Devuelve un registro o un error si no existe.	200 OK / 404 Not Found
Crear	POST /recurso	Valida el cuerpo, persiste y devuelve el registro creado.	201 Created / 400 Bad Request
Actualizar	PUT o PATCH /recurso/:id	Valida existencia y reglas; persiste cambios permitidos.	200 OK / 400 / 404
Eliminar	DELETE /recurso/:id	Elimina o desactiva según las reglas de integridad definidas.	204 No Content / 404 / 409

No es obligatorio usar PUT y PATCH simultáneamente. El equipo debe definir qué significa cada método y mantener la misma convención en toda la API.

7.2 Formato de respuestas

Respuesta exitosa de ejemplo:

```
{
  "success": true,
  "data": {
    "id": 12,
    "code": "REC-012",
    "name": "Laboratorio de sistemas",
    "status": "DISPONIBLE"
  },
  "message": "Recurso consultado correctamente"
}
```

Respuesta de error de ejemplo:

```
{
  "success": false,
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "No existe un recurso con el identificador solicitado"
  }
}
```

El formato puede adaptarse, pero debe ser consistente. No se deben enviar trazas internas, contraseñas, tokens ni detalles sensibles en las respuestas de error.

7.3 Validaciones mínimas por dominio

Usuarios: nombre y correo obligatorios; correo con formato válido; rol permitido; correo único.

Recursos: código y nombre obligatorios; tipo, ubicación y estado válidos.

Solicitudes: usuario de prueba existente; descripción no vacía; prioridad y estado dentro del catálogo; identificador único.

Reservas: usuario y recurso existentes; fecha/hora válida; fecha final posterior a la inicial; verificar disponibilidad y solapamientos.

Eventos: título, fecha, hora y lugar obligatorios; rango horario válido.

Notificaciones: destinatario de prueba válido; título y mensaje obligatorios; lectura representada de forma consistente.

7.4 Reglas del ciclo de vida de solicitudes

El Entregable 1 define el flujo: REGISTRADA → EN REVISIÓN → ASIGNADA → EN PROCESO → RESUELTA → CERRADA. En esta entrega se deberá persistir el estado y validar que las transiciones estén controladas.

Estado actual	Transición permitida de referencia	Condición
REGISTRADA	EN REVISIÓN	La solicitud existe y cumple los datos mínimos.
EN REVISIÓN	ASIGNADA	Se identifica un responsable de prueba.
ASIGNADA	EN PROCESO	El responsable inicia atención.
EN PROCESO	RESUELTA	Se registra una solución o resultado.
RESUELTA	CERRADA	Se confirma el cierre.

Si se necesita devolver una solicitud a un estado anterior o cancelar, el equipo deberá definirlo, justificarlo y probarlo. No se debe permitir que el frontend cambie libremente cualquier estado sin validación del backend.

7.5 Reglas de reservas

Una reserva debe referenciar un usuario y un recurso existentes.

El inicio debe ser anterior al final.

No se debe aceptar una reserva que se solape con otra reserva activa del mismo recurso.
El servicio debe responder con un error entendible cuando exista conflicto.
Las cancelaciones deben quedar reflejadas mediante el estado acordado.
Las comprobaciones críticas deben ejecutarse en el backend, no únicamente en React.

8. Integración entre servicios y frontend

8.1 Flujo obligatorio

- El usuario realiza una acción en una vista React, por ejemplo registrar una solicitud.
- El componente valida campos básicos y muestra el estado de envío.
- El cliente HTTP del frontend envía una petición al endpoint público del API Gateway.
- El Gateway enruta la solicitud al microservicio responsable.
- El microservicio valida la petición y aplica las reglas de negocio.
- El servicio persiste o consulta los datos en su base de datos.
- El servicio responde con JSON y un código HTTP apropiado.
- El Gateway devuelve la respuesta al frontend.
- React actualiza la interfaz y presenta confirmación o un mensaje de error.

8.2 Flujos funcionales mínimos

Flujo	Acción de demostración	Evidencia esperada
Recursos	Crear un recurso y luego consultarlo desde la lista.	Registro persistido visible en la interfaz y en la base de datos.
Solicitudes	Registrar una solicitud, consultar detalle y cambiar estado mediante transición válida.	Código de solicitud, estado y fechas recuperados desde la API.
Reservas	Consultar recursos, crear reserva y probar un horario que se solape.	Reserva guardada y conflicto rechazado por el backend.
Eventos	Crear o consultar un evento académico de prueba.	Evento visible en la vista y persistido.
Notificaciones	Generar una notificación asociada a una operación y marcarla como leída.	Cambio de estado persistido.

Errores	Consultar un ID inexistente o enviar datos incompletos.	Código HTTP y mensaje controlado; la interfaz no se bloquea.

8.3 Estados de interfaz

Cargando: indicar que la petición está en curso y evitar envíos repetidos.
Éxito: confirmar la operación y actualizar los datos mostrados.
Error de validación: indicar qué campo debe corregirse.
Error de servidor o conexión: mostrar un mensaje comprensible y permitir reintentar cuando sea apropiado.
Lista vacía: mostrar un mensaje útil, no una pantalla rota.
Detalle inexistente: presentar un estado de no encontrado.

8.4 Configuración de URLs

Las URLs base de la API deben configurarse mediante variables de entorno. El repositorio debe incluir un archivo de ejemplo sin secretos, por ejemplo VITE_API_BASE_URL=http://localhost:3000. Los puertos y nombres reales deben documentarse en el README. No se deben codificar credenciales en el frontend.

8.5 Comunicación entre microservicios

Cuando una operación requiera información de otro dominio, el equipo deberá identificar quién es dueño de cada dato y qué servicio debe consultarse. Para esta entrega se puede utilizar comunicación HTTP síncrona documentada. No se debe acceder directamente a las tablas internas de otro servicio.
Ejemplo: al registrar una reserva, Reservations Service valida el recurso y el usuario mediante los mecanismos definidos por el equipo. Si una consulta remota falla, el servicio debe devolver un error controlado y no crear una reserva inconsistente. La mensajería asíncrona puede quedar como mejora del Entregable 3, salvo que el equipo la implemente y documente.

9. Calidad, seguridad y pruebas

9.1 Requisitos de calidad

Consistencia: los mismos recursos deben representarse de manera uniforme en API, base de datos e interfaz.
Mantenibilidad: separar rutas, controladores, servicios de negocio y acceso a datos cuando corresponda.
Reutilización: evitar duplicar lógica de validación y componentes de interfaz sin necesidad.
Robustez: tratar errores de conexión, registros inexistentes y datos inválidos.
Reproducibilidad: un integrante o evaluador debe poder instalar el proyecto siguiendo instrucciones.
Claridad: nombres, mensajes, rutas y documentación deben ser comprensibles.

9.2 Seguridad básica obligatoria

Usar exclusivamente datos ficticios o de prueba.
No publicar credenciales, tokens, contraseñas ni archivos .env reales.
Validar los datos recibidos por el backend, aunque React también valide.
Usar consultas parametrizadas o el mecanismo seguro equivalente del controlador/ORM para prevenir inyección SQL.
No devolver contraseñas ni secretos en respuestas de API.
Si se implementan contraseñas, almacenar solo hashes seguros; nunca texto plano.
Separar configuración y secretos del código fuente.
Documentar el uso de CORS y limitar orígenes según el entorno de desarrollo.
Evitar que un usuario de prueba modifique recursos fuera de sus permisos si se implementan roles.

9.3 Matriz de pruebas mínima

ID	Prueba	Datos de entrada	Resultado esperado
T01	Listar recursos	GET /api/resources	200 y colección JSON.
T02	Crear recurso válido	Nombre, código, tipo y estado válidos	201; registro almacenado.
T03	Crear recurso incompleto	Omitir nombre o código	400; no se almacena.
T04	Consultar ID existente	ID válido	200; datos correctos.
T05	Consultar ID inexistente	ID que no existe	404 controlado.
T06	Actualizar recurso	Modificar ubicación o estado permitido	200; cambio persistido.
T07	Eliminar recurso	ID existente y sin conflicto de integridad	204 o respuesta documentada.

T08	Crear solicitud	Descripción y tipo válidos	201; estado inicial REGISTRADA.
T09	Transición inválida	Intentar pasar de REGISTRADA a CERRADA	400 o 409; estado no cambia.
T10	Crear reserva válida	Recurso y franja disponible	201; reserva persistida.
T11	Reserva solapada	Mismo recurso y horario conflictivo	409; no se crea reserva duplicada.
T12	Persistencia tras reinicio	Crear registro, reiniciar servicios y consultar	El registro sigue disponible.
T13	Integración UI–API	Crear desde React	La interfaz muestra la respuesta real del backend.
T14	Error de conexión	Detener temporalmente un servicio	La interfaz informa el error sin bloquearse.
T15	Health checks	GET /health de cada servicio	Respuesta de salud documentada.

9.4 Registro de defectos

Cada defecto encontrado deberá registrarse en ClickUp con identificador, descripción, pasos para reproducirlo, resultado esperado, resultado observado, severidad, responsable y evidencia. Al corregirlo, se debe enlazar el commit o Pull Request y repetir la prueba.

10. GitHub, ClickUp y trabajo colaborativo

10.1 Organización recomendada

Elemento	Convención propuesta
Repositorio	Un repositorio por equipo, con frontend, Gateway, servicios, base de datos y documentación.
Ramas estables	main para versión estable; develop para integración, si el equipo adopta este flujo.
Rama de trabajo	feature/SD2-123-persistencia-recursos
Commit	feat(resources): persist resource records [SD2-123]
Pull Request	SD2-123 – Implementar persistencia del servicio de recursos
ClickUp	Una tarea por funcionalidad verificable; incluir responsable, criterios de aceptación y evidencia.

Revisión	Un integrante distinto del autor revisa el PR antes de integrarlo cuando la organización del equipo lo permita.

10.2 Mínimos de trazabilidad por estudiante

Al menos tres tareas sustantivas asignadas en ClickUp.
Al menos cinco commits significativos relacionados con el trabajo de la entrega.
Una rama propia identificable.
Al menos dos Pull Requests o, si el tamaño del equipo no lo permite, evidencia equivalente de revisión de código acordada con la docente.
Una contribución funcional comprobable en código.
Una evidencia por tarea cerrada.
Un aporte a la documentación técnica.

Los conteos son indicadores mínimos de trazabilidad, no sustituyen la evaluación de la calidad. Se revisarán pertinencia, complejidad, funcionamiento y autoría de los aportes.

10.3 Estados de ClickUp

Backlog
Por hacer
En desarrollo
En pruebas
En revisión docente
Correcciones
Aprobado
Cerrado

10.4 Ejemplo de tarea bien formulada

Título: SD2-123 – Implementar creación y consulta de recursos con MySQL.
Descripción: implementar POST /api/resources y GET /api/resources en el servicio de Recursos, validando campos obligatorios y persistiendo la información.
Criterios de aceptación: el recurso válido se almacena; un registro incompleto devuelve 400; la consulta lista registros persistidos; las pruebas están adjuntas; el README explica cómo ejecutar el servicio.
Evidencias: enlace al PR, captura de pruebas Postman, captura del registro en la base de datos y actualización de documentación.

11. Entregables que debe subir el equipo

El equipo deberá entregar un único archivo ZIP, salvo que la docente indique otro mecanismo de recepción. El ZIP no debe incluir node_modules, credenciales ni datos personales reales.

1. Código fuente actualizado del frontend, API Gateway y seis microservicios.
2. Scripts de creación de base de datos, migraciones o mecanismo equivalente.
3. Datos semilla ficticios para reproducir los casos de prueba.
4. Archivo .env.example con nombres de variables y valores ilustrativos no sensibles.
5. Documento técnico en PDF con arquitectura, decisiones, modelo de datos, endpoints, pruebas y limitaciones.
6. Diagrama actualizado de arquitectura.
7. Diagrama entidad-relación y diccionario de datos.
8. Documentación de APIs o colección Postman exportada.
9. Matriz de pruebas con resultados y evidencias.
10. Capturas de CRUD funcionando e integración React → Gateway → microservicio → base de datos.
11. README actualizado con requisitos, instalación, configuración, ejecución y solución de problemas comunes.
12. URL del repositorio GitHub.
13. Exportación, capturas o enlace al tablero ClickUp con tareas, responsables y evidencias.
14. Matriz de aportes individuales con enlaces a tareas, ramas, commits y Pull Requests.

11.1 Estructura recomendada del paquete

```
ENTREGABLE_2_UAJS_SMART_CAMPUS.zip
├── frontend/
├── backend/
│   ├── api-gateway/
│   └── services/
├── database/
├── docs/
│   ├── Entregable_2_Tecnico.pdf
│   ├── arquitectura.png
│   ├── modelo_datos.png
│   └── matriz_pruebas.xlsx o .pdf
├── postman/
├── .env.example
└── README.md
```

11.2 Contenido mínimo del documento técnico

- Portada institucional y datos del equipo.
- Resumen del avance y alcance del Entregable 2.

- Descripción de la arquitectura y decisiones técnicas.
- Modelo de datos: diagrama, entidades, atributos y relaciones.
- Descripción de cada microservicio y sus responsabilidades.
- Catálogo de endpoints con método, ruta, parámetros, cuerpo, respuesta y errores.
- Reglas de negocio y validaciones implementadas.
- Procedimiento de instalación y ejecución.
- Pruebas ejecutadas, resultados y evidencias.
- Limitaciones conocidas y asuntos reservados para el Entregable 3.
- Enlaces a GitHub y ClickUp, con identificación de aportes individuales.

12. Rúbrica propuesta de evaluación

Criterio	Descripción	Peso
1. Modelo de datos y persistencia	Modelo coherente, restricciones, scripts reproducibles y persistencia real.	15 %
2. CRUD de microservicios	Operaciones completas y funcionales en los dominios requeridos.	20 %
3. Reglas de negocio y validaciones	Validación en backend, reglas de solicitudes y reservas, integridad.	15 %
4. API Gateway e integración	Rutas correctas y comunicación entre frontend, Gateway y servicios.	15 %
5. Pruebas y manejo de errores	Matriz de pruebas, respuestas HTTP, errores controlados y evidencia.	10 %
6. Calidad y seguridad básica	Configuración segura, consultas parametrizadas, datos ficticios y estructura mantenible.	10 %
7. Documentación técnica	README, modelo de datos, catálogo de APIs, instrucciones y evidencias.	5 %
8. GitHub, ClickUp y aporte individual	Tareas, ramas, commits, PR, evidencias y contribución demostrable.	10 %
TOTAL		100 %